

Контейнеризація

I Команди Docker

1. Встановіть Docker на свою систему (якщо він ще не встановлений) та перевірте його статус.

2. Завантажте офіційний Docker образ, наприклад, Ubuntu, з Docker Hub.

```
docker pull ubuntu
```

3. Запустіть контейнер на основі завантаженого образу і відкрийте оболонку контейнера:

```
docker run -it --name my-ubuntu-container ubuntu /bin/bash
```

4. У контейнері виконайте наступну команду, щоб встановити веб-сервер Apache:

```
apt-get update
```

```
apt-get install -y apache2
```

5. Запустіть веб-сервер Apache в контейнері:

```
apache2ctl start
```

6. Вийдіть із контейнера, але не зупиняйте його:

```
exit
```

7. Переконайтеся, що контейнер запущений:

```
docker ps -a
```

8. Відкрийте веб-браузер і перейдіть за адресою <http://localhost> або <http://IP-адреса-системи>. Ви повинні бачити сторінку Apache за замовчуванням.

9. Зупиніть контейнер:

```
docker stop my-ubuntu-container
```

10. Видаліть контейнер:

```
docker rm my-ubuntu-container
```

ЗАВДАННЯ 1

Повторіть попередні дії, але замініть Ubuntu на Alpine Linux, а apache на nginx

II Docker Volumes

Docker Volumes

Кожний контейнер створюється з визначення образу кожного разу при запуску. Хоча контейнери можуть створювати, оновлювати та видаляти файли, всі зміни втрачаються, коли видаляєте контейнер, і Docker ізолює всі зміни до цього контейнера. За допомогою volumes можливо змінити цю ситуацію.

Volumes надають можливість підключати конкретні шляхи файлової системи контейнера назад до машини-хоста. Якщо ви монтуєте каталог в контейнері, зміни в цьому каталозі також будуть видно на машині-хості. Якщо ви монтуєте той самий каталог при перезапуску контейнера, ви побачите ті ж самі файли.

Збереження даних MySQL

За замовчуванням mysql зберігає дані в за шляхом /var/lib/mysql. Якщо зберегти файли данної директорії на хості і зробити їх доступним для наступного контейнера, то дані не будуть втраченими після зупинки контейнеру. Створивши Volumes і прикріпивши його (часто називається "монтуванням") до каталогу, де ви зберегли дані, ви можете зберегти дані. Коли ваш контейнер записує файли у директорії /var/lib/mysql, він зберігає дані на хості в Volumes.

Створення Volumes та запуск контейнеру

Створити Volumes та запустити контейнер можливо за допомогою командного рядка або графічного інтерфейсу Docker Desktop.

Створіть об'єм за допомогою команди `docker volume create`.

```
docker volume create mysql-db
```

```
docker volumes inspect mysql-db
```

Запустіть контейнер mysql та додайте параметр `--mount`, щоб вказати монтування Volumes. Надайте Volumes ім'я і приєднайте його до каталогу /var/lib/mysql в контейнері, який захоплює всі файли, створені за цим шляхом.

```
docker run --name mysql -e MYSQL_ROOT_PASSWORD=123456 -e MYSQL_DATABASE=test_db -e MYSQL_USER=mysql -d -p 3306:3306 --mount source=mysql-db,target=/var/lib/mysql mysql:latest
```

Якщо потрібно встановити, де Docker зберігає дані, ви можете скористатися командою:

```
docker volume inspect mysql-db
[
  {
    "CreatedAt": "2023-09-28T13:32:35Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mysql-db/_data",
    "Name": "mysql-db",
    "Options": null,
    "Scope": "local"
  }
]
```

III Bind mounts

Bind mounts — це ще один тип монтування, який дозволяє надати спільний доступ до каталогу з файлової системи хоста в контейнер. рацюючи над програмою, ви можете використовувати Bind mounts, щоб змонтувати вихідний код у контейнер. Контейнер одразу бачить зміни, які ви вносите в код, щойно ви зберігаєте файл. Це означає, що ви можете запускати процеси в контейнері, які спостерігають за змінами файлової системи та реагують на них.

У наведеній нижче таблиці наведено основні відмінності між volumes та bind mounts.

	Named volumes	Bind mounts
Host location	Docker chooses	You decide
Mount example (using <code>--mount</code>)	<code>type=volume,src=my-volume,target=/usr/local/data</code>	<code>type=bind,src=/path/to/data,target=/usr/local/data</code>
Populates new volume with container contents	Yes	No
Supports Volume Drivers	Yes	No

Для запуску попереднього прикладу використовуйте команду:

```
docker run --name mysql -e MYSQL_ROOT_PASSWORD=123456 -e MYSQL_DATABASE=test_db -e
MYSQL_USER=mysql -d -p 3306:3306 --mount type=bind src=<local-
patch>/db,target=/var/lib/mysql mysql:latest
```

або у альтернативному вигляді:

```
docker run --name mysql -e MYSQL_ROOT_PASSWORD=123456 -e MYSQL_DATABASE=test_db -e
MYSQL_USER=mysql -d -p 3306:3306 -v <local-patch>/db:/var/lib/mysql mysql:latest
```

Завдання 2

1. Створіть Docker volume за допомогою команди:

```
docker volume create my_data_volume
```

2. Запустіть контейнер створений під час попередньої роботи, монтуючи створений Docker volume:

Приклад для контейнеру, що базується на Ubuntu

```
docker run -it --name my_container -v my_data_volume:/app ubuntu
```

3. В контейнері перейдіть в директорію **/app** та створіть файл або папку. Переконайтеся, що дані зберігаються у volume.

4. Виконайте запис у створений Docker volume зсередини контейнера.

```
echo "Hello, Docker!" > /app/myfile.txt
```

5. Запустіть другий контейнер та монтуйте той самий Docker volume.

```
docker run -it --name my_other_container -v my_data_volume:/app ubuntu
```

6. Переконайтеся, що дані, які ви записали в volume з першого контейнера, доступні в другому контейнері.

7. Змініть права доступу до файлів та папок в volume так, щоб тільки конкретні користувачі мали доступ до них

```
chmod 700 /app/file_in_volume
```

IV Dockerfiles

Завдання 3

1. Створіть новий каталог для лабораторної роботи та перейдіть в нього.

```
mkdir my_docker_lab
cd my_docker_lab
```

2. Створіть Dockerfile у цьому каталозі з вмістом, що базується на образі Alpine та встановлює веб-сервер Nginx:

```
# Використання базового образу Alpine
FROM alpine
```

```
# Встановлення пакетів Nginx та supervisord
RUN apk update && apk add nginx supervisor
```

```
# Копіювання конфігураційних файлів в образ
COPY nginx.conf /etc/nginx/nginx.conf
COPY supervisord.conf /etc/supervisord.conf
```

```
# Вказання команд, які будуть виконані при старті контейнера
```

CMD ["supervisord", "-c", "/etc/supervisord.conf"]

3. Створіть конфігураційний файл `nginx.conf` для Nginx та `supervisord.conf` для supervisord у тому ж каталозі:

Nginx

```
worker_processes 1;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    server {
        listen 80;
        server_name localhost;

        location / {
            root /usr/share/nginx/html;
            index index.html;
        }
    }
}
```

supervisord.conf

```
[supervisord]
nodaemon=true

[program:nginx]
command=nginx -g "daemon off;"
autorestart=true
```

4. Побудуйте Docker образ за допомогою Dockerfile:

```
docker build -t my-nginx-image .
```

5. Запустіть контейнер і перевірте, що налаштування та програми з Dockerfile правильно встановлені в контейнері:

```
docker run -d --name my-nginx-container -p 8080:80 my-nginx-image
```

Перейдіть за адресою `http://localhost:8080` у веб-браузері та переконайтеся, що веб-сервер Nginx працює.

6. Зупиніть та видаліть контейнер, а також видаліть Docker образ:

```
docker stop my-nginx-container
docker rm my-nginx-container
docker rmi my-nginx-image
```

7. Додайте параметри до Dockerfile для передачі аргументів під час побудови образу. Додайте аргумент `NGINX\_VERSION` для визначення версії Nginx.

```
ARG NGINX_VERSION=latest
FROM alpine:${NGINX_VERSION}
```

Решта Dockerfile

8. Побудуйте новий Docker образ, використовуючи аргументи:

```
docker build -t my-nginx-image --build-arg NGINX_VERSION=1.20.1 .
```

9. Після побудови образу переконайтеся, що аргументи були правильно передані та використані у Dockerfile.

10. Створіть нову папку та в ній створіть новий Dockerfile, що базується на образі Ubuntu та встановлює веб-сервер Apache2.

V Docker Networking

Налаштування Docker контейнера з мережевими можливостями за допомогою Dockerfile

Завдання 4

1. Створіть новий каталог для лабораторної роботи та перейдіть в нього.

```
mkdir dockerfile_networking_lab  
cd dockerfile_networking_lab
```

2. Створіть Dockerfile у цьому каталозі для створення Docker образу на базі Alpine.

```
# Використання базового образу Alpine  
FROM alpine
```

```
# Встановлення пакетів, необхідних для роботи з мережевими можливостями  
RUN apk update && apk add iproute2 curl
```

3. Додайте команди до Dockerfile для налаштування мережі в контейнері. Наприклад, встановлення IP-адреси та маршрутизації.

```
# Решта Dockerfile
```

```
# Налаштування мережі  
RUN ip addr add 192.168.0.2/24 dev eth0  
RUN ip route add default via 192.168.0.1
```

4. Побудуйте Docker образ за допомогою Dockerfile:

```
docker build -t networking-image .
```

5. Запустіть контейнер на основі створеного образу та перевірте налаштування мережі:

```
docker run -it --name networking-container networking-image /bin/sh
```

У контейнері виконайте команди:

```
# Перевірка IP-адреси  
ip addr show eth0
```

```
# Перевірка маршруту за замовчуванням  
ip route show
```

6. Вийдіть із контейнера та зупиніть його:

```
exit
```

```
docker stop networking-container
```

7. Видаліть контейнер та Docker образ:

```
docker rm networking-container  
docker rmi networking-image
```

8. Додайте параметри до Dockerfile для передачі аргументів під час побудови образу. Наприклад, додайте аргумент `IP\_ADDRESS` для встановлення IP-адреси.

```
ARG IP_ADDRESS=192.168.0.2  
FROM alpine
```

```
# Встановлення пакетів та налаштування мережі  
RUN apk update && apk add iproute2 curl  
RUN ip addr add ${IP_ADDRESS}/24 dev eth0
```

9. Побудуйте новий Docker образ, використовуючи аргументи:

```
docker build -t networking-image --build-arg IP_ADDRESS=192.168.1.2 .
```

10. Після побудови образу переконайтеся, що аргументи були правильно передані та використані у Dockerfile.